SEV 측정 및 검증

개요

AMD SEV 측정 및 검증은 게스트 가상 머신의 무결성을 보장하는 핵심 보안 메커니즘입니다. 이 문서는 게스트 이미지 측정 프로세스, OVMF, VMSA, GCTX 측정 방법, 해시 체인 생성 및 무결성 검증 과정을 상세히 설명합니다.

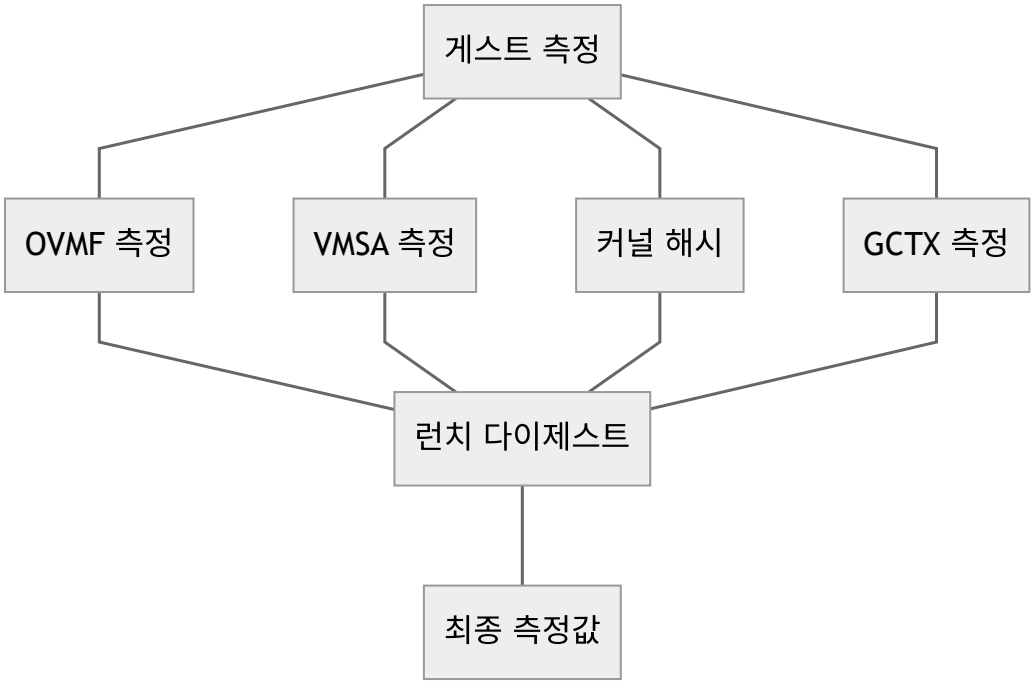
측정 프로세스 개요

1. 측정의 목적

SEV 측정은 다음과 같은 목적을 가집니다:

- **무결성 보장**: 게스트 이미지가 변조되지 않았음을 확인
- **신뢰성 검증**: 예상된 소프트웨어가 실행되고 있음을 증명
- **원격 증명**: 원격 당사자가 게스트 상태를 검증할 수 있도록 지원

2. 측정 구성 요소



OVMF 측정

1. OVMF 개요

OVMF (Open Virtual Machine Firmware)는 가상 머신의 UEFI 펌웨어입니다.

```
use sev::measurement::ovmf::OVMF;
use std::path::PathBuf;

// OVMF 파일 로드
let ovmf_file = PathBuf::from("path/to/OVMF.fd");
let ovmf = OVMF::new(ovmf_file)?;

// OVMF 데이터 및 GPA 확인
let ovmf_data = ovmf.data();
```

```
let ovmf_gpa = ovmf.gpa(); // 4GB - OVMF 크기
```

2. OVMF 메타데이터 파싱

OVMF는 SEV 관련 메타데이터를 포함합니다:

```
use sev::measurement::ovmf::SectionType;


// 메타데이터 항목 확인
let metadata_items = ovmf.metadata_items();


for item in metadata_items {
match item.section_type {
SectionType::SnpSecMemory => {
println!("SNP 보안 메모리: GPA=0x{:x}, Size=0x{:x}",
item.gpa, item.size);
},
SectionType::SnpSecrets => {
println!("SNP 비밀 영역: GPA=0x{:x}", item.gpa);
},
SectionType::Cpuid => {
println!("CPUID 페이지: GPA=0x{:x}", item.gpa);
},
SectionType::SnpKernelHashes => {
```

```
println!("커널 해시 영역: GPA=0x{:x}, Size=0x{:x}",
item.gpa, item.size);
},
SectionType::SvsmCaa => {
println!("SVSM CAA 영역: GPA=0x{:x}", item.gpa);
},
}
}
```

3. OVMF 테이블 파싱

```
// SEV 해시 테이블 지원 확인
if ovmf.is_sev_hashes_table_supported() {
let sev_hashes_gpa = ovmf.sev_hashes_table_gpa()?;
println!("SEV 해시 테이블 GPA: 0x{:x}", sev_hashes_gpa);
}

// SEV-ES 리셋 EIP 확인
let reset_eip = ovmf.sev_es_reset_eip()?;
println!("SEV-ES 리셋 EIP: 0x{:x}", reset_eip);
```

VMSA 측정

1. VMSA 개요

VMSA (Virtual Machine Save Area)는 SEV-ES에서 vCPU 상태를 저장하는 영역입니다.

```
use sev::measurement::vmsa::{VMSA, VMMType, GuestFeatures};
use sev::measurement::vcpu_types::CpuType;

// VMSA 생성
let vmsa = VMSA::new(
reset_eip, // AP EIP
CpuType::EpycV1, // CPU 타입
VMMType::QEMU, // VMM 타입
Some(vcpu_count), // vCPU 수
GuestFeatures::default(), // 게스트 기능
);
```

2. VMSA 페이지 생성

```
// vCPU 수에 따른 VMSA 페이지 생성
let vcpu_count = 4;
let vmsa_pages = vmsa.pages(vcpu_count)?;

for (i, page) in vmsa_pages.iter().enumerate() {
println!("VMSA 페이지 {}: {} 바이트", i, page.len());
```

```
// 각 페이지는 4096 바이트
assert_eq!(page.len(), 4096);
}
```

3. 게스트 기능 설정

```
use sev::measurement::vmsa::GuestFeatures;

// SNP 기능 활성화
let mut guest_features = GuestFeatures::default();
// SNP 활성화 (비트 0)
let guest_features = GuestFeatures(0x1);

// 추가 기능들
let advanced_features = GuestFeatures(
0x1 | // SNP Active
0x2 | // vTOM
0x4 | // Reflect VC
0x200 // Secure TSC
);
```

4. VMM별 차이점

```
// VMM 타입별 VMSA 설정 차이
match vmm_type {
VMMType::QEMU => {
// QEMU 특화 설정
// - CS 플래그: 0x9b
// - SS 플래그: 0x93
// - MXCSR: 0x1f80
},
VMMType::EC2 => {
// AWS EC2 특화 설정
// - 다른 플래그 값 사용
},
VMMType::KRUN => {
// KRUN 특화 설정
// - 특별한 메모리 레이아웃
},
}
```

GCTX 측정 (SEV-SNP)

1. GCTX 개요

GCTX (Guest Context)는 SEV-SNP에서 게스트의 전체 상태를 추적하는 구조입니다.

```rust
use sev::measurement::gctx::Gctx;
use sev::launch::snp::PageType;

// GCTX 초기화 (기본값: 모든 0)
let mut gctx = Gctx::default();

// 또는 기존 시드로 초기화
let seed = &[0u8; 48]; // 48바이트 런치 다이제스트
let mut gctx = Gctx::new(seed)?;
```

2. 페이지 업데이트

```rust
// 일반 데이터 페이지 업데이트
let guest_data = &[/* 게스트 데이터 */];
let gpa = 0x1000; // 게스트 물리 주소

gctx.update_page(
PageType::Normal,
gpa,
Some(guest_data),
None
)?;
```

```
// 제로 페이지 업데이트
let zero_size = 4096;
gctx.update_page(
PageType::Zero,
gpa,
None,
Some(zero_size)
)?;

// VMSA 페이지 업데이트
gctx.update_page(
PageType::Vmsa,
0xFFFFFFFFF000, // VMSA GPA
Some(&vmsa_page),
None
)?;
```

3. 페이지 타입별 처리

```
use sev::launch::snp::PageType;

// 페이지 타입별 측정 방법
match page_type {
```

```
PageType::Normal => {
// 실제 데이터 해시 계산
// 4KB 단위로 SHA-384 해시
},
PageType::Vmsa => {
// VMSA 데이터 해시
// 고정 GPA 사용 (0xFFFFFFFFF000)
},
PageType::Zero => {
// 제로 해시 사용
// 실제 데이터 없이 크기만 지정
},
PageType::Unmeasured => {
// 측정되지 않는 페이지
// 제로 해시 사용
},
PageType::Secrets => {
// 비밀 저장 페이지
// 제로 해시 사용
},
PageType::Cpuid => {
// CPUID 정보 페이지
// 제로 해시 사용
```

```
},

}
```

4. 런치 다이제스트 완료

```
// GCTX 업데이트 완료
let  completed_gctx = gctx.finished();


// 최종 런치 다이제스트 획득
let  launch_digest = completed_gctx.ld();
println!("런치 다이제스트: {}", launch_digest.get_hex_ld());
```

SEV 해시 계산

1. 커널 해시

```
use sev::measurement::sev_hashes::SevHashes;
use std::path::PathBuf;


// 커널, initrd, 커맨드라인 해시 생성
let  kernel_path = PathBuf::from("path/to/kernel");
let  initrd_path = Some(PathBuf::from("path/to/initrd"));
let  cmdline = Some("console=ttyS0 root=/dev/sda1");
```

```rust
let sev_hashes = SevHashes::new(kernel_path, initrd_path, cmdline)?;
```

2. 해시 테이블 구성

```rust
// SEV 해시 테이블 생성
let hash_table = sev_hashes.construct_table()?;

// 해시 페이지 생성 (4KB 페이지 내 특정 오프셋)
let offset = 0x100; // 페이지 내 오프셋
let hash_page = sev_hashes.construct_page(offset)?;

assert_eq!(hash_page.len(), 4096);
```

3. 해시 테이블 구조

SEV 해시 테이블은 다음 GUID들을 사용합니다:

```rust
// 해시 테이블 헤더 GUID
const SEV_HASH_TABLE_HEADER_GUID: &str = "9438d606-4f22-4cc9-b479-a793d411fd21";

// 개별 항목 GUID들
const SEV_KERNEL_ENTRY_GUID: &str = "4de79437-abd2-427f-b835-d5b172d2045b";
const SEV_INITRD_ENTRY_GUID: &str = "44baf731-3a2f-4bd7-9af1-41e29169781d";
```

```
const SEV_CMDLINE_ENTRY_GUID: &str = "97d02dd8-bd20-4c94-aa78-e7714d36ab2a";
```

측정 계산 예제

1. SEV 측정 계산

```
use sev::measurement::sev::{sev_calc_launch_digest, SevMeasurementArgs};

// SEV 측정 인수 구성
let  sev_args = SevMeasurementArgs {
ovmf_file: PathBuf::from("OVMF.fd"),
kernel_file: Some(PathBuf::from("vmlinuz")),
initrd_file: Some(PathBuf::from("initrd.img")),
append: Some("console=ttyS0"),
};

// SEV 런치 다이제스트 계산
let  launch_digest = sev_calc_launch_digest(sev_args)?;
println!("SEV 측정값: {}", hex::encode(launch_digest));
```

2. SEV-ES 측정 계산

```
use sev::measurement::sev::{seves_calc_launch_digest, SevEsMeasurementArgs};
use sev::measurement::vcpu_types::CpuType;
```

```rust
use sev::measurement::vmsa::VMMType;


// SEV-ES 측정 인수 구성
let  seves_args = SevEsMeasurementArgs {
vcpus: 2,
vcpu_type: CpuType::EpycV1,
ovmf_file: PathBuf::from("OVMF.fd"),
kernel_file: Some(PathBuf::from("vmlinuz")),
initrd_file: Some(PathBuf::from("initrd.img")),
append: Some("console=ttyS0"),
vmm_type: Some(VMMType::QEMU),
};


// SEV-ES 런치 다이제스트 계산
let  launch_digest = seves_calc_launch_digest(seves_args)?;
println!("SEV-ES 측정값: {}", hex::encode(launch_digest));
```

3. SEV-SNP 측정 계산

```rust
use sev::measurement::snp::{snp_calc_launch_digest, SnpMeasurementArgs};
use sev::measurement::vmsa::GuestFeatures;


// SEV-SNP 측정 인수 구성
```

```rust
let snp_args = SnpMeasurementArgs {
vcpus: 4,
vcpu_type: CpuType::EpycV4,
ovmf_file: PathBuf::from("OVMF.fd"),
guest_features: GuestFeatures::default(),
kernel_file: Some(PathBuf::from("vmlinuz")),
initrd_file: Some(PathBuf::from("initrd.img")),
append: Some("console=ttyS0"),
ovmf_hash_str: None, // 또는 미리 계산된 OVMF 해시
vmm_type: Some(VMMType::QEMU),
};

// SEV-SNP 런치 다이제스트 계산
let launch_digest = snp_calc_launch_digest(snp_args)?;
println!("SEV-SNP 측정값: {}", launch_digest.get_hex_ld());
```

4. OVMF 해시만 계산

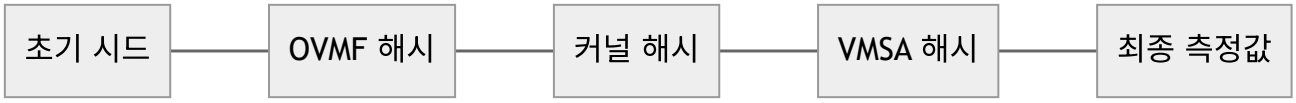
```rust
use sev::measurement::snp::calc_snp_ovmf_hash;

// OVMF 해시만 별도 계산
let ovmf_hash = calc_snp_ovmf_hash(PathBuf::from("OVMF.fd"))?;
```

```
println!("OVMF 해시: {}", ovmf_hash.get_hex_ld());
```

해시 체인 생성 및 검증

1. 해시 체인 구조

SEV 측정은 다음과 같은 해시 체인을 형성합니다:



2. 단계별 해시 계산

```
use openssl::sha::{Sha256, sha384};

// SEV의 경우 SHA-256 사용
let mut hasher = Sha256::new();
hasher.update(ovmf_data);
hasher.update(kernel_hashes);
hasher.update(vmsa_data);
let sev_digest = hasher.finish();

// SEV-SNP의 경우 SHA-384 사용
let snp_digest = sha384(&page_info);
```

3. 측정값 검증

```
// 예상 측정값과 실제 측정값 비교
fn verify_measurement(
expected: &[u8],
actual: &[u8]
) -> Result<bool, MeasurementError> {
if expected.len() != actual.len() {
return Ok(false);
}
// 상수 시간 비교 (타이밍 공격 방지)
let mut result = 0u8;
for (e, a) in expected.iter().zip(actual.iter()) {
result |= e ^ a;
}
Ok(result == 0)
}

// 사용 예제
let expected_measurement = hex::decode("abcd...")?;
let actual_measurement = snp_calc_launch_digest(args)?;

if verify_measurement(&expected_measurement, &actual_measurement.try_into()?)?
println!("측정값 검증 성공");
```

```
} else {
println!("측정값 검증 실패");
}
```

무결성 검증 과정

1. 런치 시 검증

```
// 런치 과정에서 측정값 확인
let launcher = launcher.measure()?;
let measurement = launcher.measurement();

// 예상 측정값과 비교
let expected = calculate_expected_measurement(&guest_config)?;
if measurement.measure != expected {
return Err("측정값 불일치".into());
}
```

2. 원격 증명 검증

```
// 원격 증명 보고서에서 측정값 추출
let attestation_report = launcher.report(nonce)?;
let reported_measurement = extract_measurement_from_report(&attestation_report)
```

```
// 독립적으로 계산한 측정값과 비교
let calculated_measurement = snp_calc_launch_digest(args)?;
if reported_measurement != calculated_measurement.try_into()? {
return Err("원격 증명 검증 실패".into());
}
```

3. 지속적 무결성 모니터링

```
// 런타임 중 무결성 확인
fn monitor_guest_integrity(handle: &Handle) -> Result<(), Error> {
loop {
// 주기적으로 게스트 상태 확인
let current_state = get_guest_state(handle)?;
// 예상 상태와 비교
if !verify_guest_state(&current_state)? {
return Err("게스트 무결성 위반 감지".into());
}
std::thread::sleep(std::time::Duration::from_secs(60));
}
}
```

오류 처리 및 디버깅

1. 일반적인 측정 오류

```
use sev::error::MeasurementError;

match  measurement_result {
Err(MeasurementError::InvalidOvmfKernelError) => {
eprintln!("OVMF가 커널 해시를 지원하지 않습니다");
// OVMF 파일 확인 또는 커널 없이 측정
},
Err(MeasurementError::MissingSection(section)) => {
eprintln!("필수 섹션이 누락되었습니다: {}", section);
// OVMF 메타데이터 확인
},
Err(MeasurementError::InvalidSevModeError(mode)) => {
eprintln!("지원되지 않는 SEV 모드: {}", mode);
// 올바른 SEV 모드 지정
},
Ok(measurement) => {
println!("측정 성공: {}", hex::encode(measurement));
},
}
```

2. 디버깅 도구

```rust
// 상세 측정 로그 활성화
#[cfg(debug_assertions)]
fn debug_measurement_process(args: &SnpMeasurementArgs) {
println!("=== SEV-SNP 측정 디버그 정보 ===");
println!("vCPU 수: {}", args.vcpus);
println!("CPU 타입: {:?}", args.vcpu_type);
println!("게스트 기능: {:?}", args.guest_features);
if let Some(kernel) = &args.kernel_file {
println!("커널 파일: {:?}", kernel);
}
if let Some(initrd) = &args.initrd_file {
println!("Initrd 파일: {:?}", initrd);
}
if let Some(append) = args.append {
println!("커널 매개변수: {}", append);
}
}
```

3. 측정값 분석

```rust
// 측정값 구성 요소별 분석
fn analyze_measurement_components(args: &SnpMeasurementArgs) -> Result<(), Erro
// OVMF 해시 계산
```

```
let ovmf_hash = calc_snp_ovmf_hash(args.ovmf_file.clone())?;
println!("OVMF 해시: {}", ovmf_hash.get_hex_ld());
// 커널 해시 계산 (있는 경우)
if let Some(kernel) = &args.kernel_file {
let sev_hashes = SevHashes::new(
kernel.clone(),
args.initrd_file.clone(),
args.append
)?;
let hash_table = sev_hashes.construct_table()?;
println!("커널 해시 테이블: {}", hex::encode(hash_table));
}
// VMSA 해시 계산
let vmsa = VMSA::new(
0xffffff0,
args.vcpu_type,
args.vmm_type.unwrap_or(VMMType::QEMU),
Some(args.vcpus as u64),
args.guest_features,
);
let vmsa_pages = vmsa.pages(args.vcpus as usize)?;
for (i, page) in vmsa_pages.iter().enumerate() {
let page_hash = sha384(page);
```

```
    println!("VMSA 페이지 {} 해시: {}", i, hex::encode(page_hash));
}
    Ok(())
}
```

모범 사례

1. 보안 고려사항

1. **측정값 보호**: 측정값을 안전하게 저장하고 전송하세요
2. **타이밍 공격 방지**: 상수 시간 비교를 사용하세요
3. **재현 가능성**: 동일한 입력에 대해 동일한 측정값이 나오는지 확인하세요
4. **검증 체인**: 전체 검증 체인을 구현하세요

2. 성능 최적화

1. **해시 캐싱**: 자주 사용되는 해시값을 캐시하세요
2. **병렬 처리**: 독립적인 측정 작업을 병렬로 수행하세요
3. **메모리 효율성**: 대용량 파일 처리 시 스트리밍을 사용하세요

3. 호환성 고려사항

1. **VMM 차이점**: 각 VMM의 특성을 고려하세요
2. **CPU 세대**: 대상 CPU 세대의 기능을 확인하세요
3. **펌웨어 버전**: OVMF 버전 호환성을 확인하세요

결론

SEV 측정 및 검증은 보안 가상화의 핵심 요소입니다. OVMF, VMSA, GCTX 등 각 구성 요소의 측정 방법을 이해하고, 올바른 해시 체인을 구성하여 게스트의 무결성을 보장할 수 있습니다. 특히 SEV-SNP에서는 더욱 정교한 측정 메커니즘을 통해 강화된 보안을 제공합니다.